PropOS Python 虚拟环境（`.venv`）使用指南

概述

`.venv` 是项目根目录下的 **Python 3.14 虚拟环境**，专用于驱动 PropOS 的文档生成工具链（Markdown → Word → PDF 转换流水线）。它与项目的 Dart/Flutter 主业务代码完全隔离，不影响后端或移动端的运行时。

> 注意：`.venv` 目录不纳入 Git 版本控制（应在 `.gitignore` 中排除）。新成员须按本文档完成本地初始化。

为什么需要独立虚拟环境

问题	解决方案
文档工具依赖的 `python-docx`、`ezdxf` 等包版本与系统 Python 环境可能冲突	通过 `venv` 隔离，不污染全局 Python
`scripts/md_to_pdf.sh` 需要确定性的可执行文件路径	脚本优先使用 `.venv/bin/python3`，路径固定，无需手动激活
CI 或其他开发者机器无需提前安装这些工具包	通过 `requirements.txt`（或手动 pip install）一次性复现

目录结构说明

```
.venv/
├── bin/                    # 可执行文件
│   ├── python3             # Python 3.14 解释器（符号链接）
│   ├── pip / pip3          # 包管理器
│   ├── activate            # bash/zsh 激活脚本
│   ├── activate.fish       # fish shell 激活脚本
│   ├── activate.csh        # csh 激活脚本
│   ├── Activate.ps1        # PowerShell 激活脚本（Windows）
│   ├── ezdxf               # CAD DXF 命令行工具（由 ezdxf 包提供）
│   ├── pymupdf             # PDF 操作命令行工具（由 PyMuPDF 提供）
│   └── pyside6-*           # PySide6 GUI 工具套件（matplotlib 渲染后端）
├── lib/
│   └── python3.14/
│       └── site-packages/  # 所有已安装的第三方包
├── include/                # C 扩展编译头文件
├── share/                  # 共享资源（字体等）
└── pyvenv.cfg              # 虚拟环境元数据（Python 版本、主程序路径）
```

已安装的核心依赖包

包名	版本	用途
`python-docx`	1.2.0	`md2word.py` 核心依赖：将 Markdown 渲染为 Word `.docx` 文档，支持标题、表格、代码块、列表、行内格式等
`ezdxf`	1.4.3	CAD DXF 文件解析/处理（M1 资产模块 CAD 转换预留）；亦提供命令行工具 `ezdxf`
`PyMuPDF`	1.27.2	PDF 文件读写与操作（`pymupdf` 命令行工具）
`matplotlib`	3.10.8	图表渲染（文档中嵌入数据可视化图表时使用）
`numpy`	2.4.4	数值计算（matplotlib / ezdxf 的底层依赖）
`Pillow`	12.2.0	图像处理（文档中嵌入图片时的格式转换）
`lxml`	6.0.2	XML/HTML 解析（python-docx 底层依赖）
`fonttools`	4.62.1	字体文件操作（文档字体嵌入处理）
`PySide6`	6.11.0	Qt6 Python 绑定（matplotlib PySide6 后端渲染）

与项目脚本的关联

`.venv` 被以下脚本直接调用：

1. `scripts/md_to_pdf.sh`（主入口）

```
# 脚本内部逻辑：优先使用 .venv，兜底使用系统 python3
VENV_PYTHON="$WORKSPACE/.venv/bin/python3"
if [[ -x "$VENV_PYTHON" ]]; then
    PYTHON="$VENV_PYTHON"
else
    PYTHON="python3"
fi
```

调用链：

```
md_to_pdf.sh
  └→ .venv/bin/python3 scripts/md2word.py   <input.md>  <output.docx>
  └→ .venv/bin/python3 scripts/docx2pdf.py  <input.docx> <output.pdf>
  └→ rm <output.docx>   # 删除中间文件
```

2. `scripts/md2word.py`

依赖 `python-docx` 将 Markdown 转换为格式化 Word 文档。不可在系统 Python 中运行（除非全局已安装 `python-docx` ）。

3. `scripts/docx2pdf.py`

通过 macOS AppleScript 驱动 **Pages.app** 完成 `.docx → .pdf` 转换。本脚本本身无额外 pip 依赖，仅要求 macOS 系统安装了 Pages.app。

4. `scripts/check_env.sh`

环境就绪检查脚本会验证 `.venv` 中 `python-docx` 和 `ezdxf` 是否可用：

```
bash scripts/check_env.sh
```

初始化（首次搭建）

```
# 1. 使用 Python 3.14 创建虚拟环境（需先通过 Homebrew 安装 Python 3.14）
python3.14 -m venv .venv

# 2. 激活环境
source .venv/bin/activate

# 3. 安装所有依赖
pip install python-docx ezdxf PyMuPDF matplotlib numpy Pillow lxml fonttools PySide6

# 4. 验证
python3 -m docx; python3 -c "import docx, ezdxf; print('OK')"
```

> 如果机器上没有 Python 3.14，可使用 `brew install python@3.14` 安装。
> 3.10+ 版本均兼容，修改创建命令中的版本号即可。

日常使用

方式一：直接使用脚本（推荐，无需手动激活）

```
# 转换单个文档
bash scripts/md_to_pdf.sh docs/backend/API_INVENTORY_v1.7.md

# 批量转换
bash scripts/md_to_pdf.sh docs/backend/*.md
```

脚本自动调用 `.venv/bin/python3` ，无需提前 `**source activate**`。

方式二：激活后交互使用

```
# 激活虚拟环境
source .venv/bin/activate

# 确认激活（命令提示符前会出现 (.venv) 前缀）
which python3   # → /Users/wanxt/app/propos/.venv/bin/python3

# 手动执行转换
python3 scripts/md2word.py docs/backend/API_INVENTORY_v1.7.md /tmp/output.docx
python3 scripts/docx2pdf.py /tmp/output.docx pdfdocs/backend/output.pdf

# 退出虚拟环境
deactivate
```

注意事项

场景	说明
Windows 开发者	`docx2pdf.py` 依赖 macOS Pages.app，Windows 上需替换为其他 DOCX→PDF 后端（如 LibreOffice）
CI/CD 环境	在 CI 中初始化 `.venv` 后执行批量转换；若无 Pages.app，可考虑 `pandoc + wkhtmltopdf` 作为替代
包版本升级	升级前在 `.venv` 中测试，确认 `md2word.py` 渲染结果无变化后再更新
ezdxf 扩展用途	M1 模块 CAD→SVG/PNG 转换功能正式开发时，将在 `.venv` 中调用 `ezdxf` Python API
不要 **commit `.venv`**	虚拟环境目录体积大（约数百 MB）且包含平台相关二进制，应在 `.gitignore` 中排除

与项目其他工具链的边界

```
PropOS 工具链全景
│
├── Dart SDK / Flutter   ← 主业务代码（后端 + 移动端 + Web）
│   └── pub.dev 包依赖
│
├── PostgreSQL           ← 数据存储
│
└── .venv（Python）    ← 仅限文档工具链
    ├── md → docx        (python-docx)
    ├── docx → pdf       (macOS Pages AppleScript)
    ├── CAD DXF 预处理 (ezdxf，M1 资产模块)
    └── 图表/图像处理  (matplotlib + Pillow)
```

`.venv` 中的 Python 代码不参与任何业务逻辑，与 Dart/Flutter 代码库完全解耦。